Deep Learning

Homework 3

Iman Jamshidi

Fall2025

Step 1: Data

1) **How many data samples are included in the dataset?**
   100 samples total

- 80 training images
- 20 test images

2) **Which problem will this dataset try to address?**
   This project focuses on retinal blood vessel segmentation, which helps automate the detection and diagnosis of eye diseases such as diabetic retinopathy, glaucoma, and hypertension. In this medical image segmentation task, each pixel in the retinal image is classified into one of two categories: vessel or background.

3) **What is the dimension ranging in the dataset?**
   The original retinal images have variable dimensions, as is common in medical datasets. For consistency during training, all images are resized to 256×256×3 (RGB format). The model's output is a 256×256×1 binary mask, where each pixel represents whether it belongs to a blood vessel (1) or the background (0).

4) **Does the dataset have any missing information?**
   There is no missing information in the dataset. Each of the 100 retinal images has a corresponding ground truth mask that accurately represents the vessel structures, ensuring complete and consistent data for training and evaluation.

5) **What is the label of this dataset?**
   The dataset labels are binary segmentation masks, where each pixel is classified as either background or blood vessel. Specifically:
   - 0 (black) represents the background
   - 1 (white) represents the blood vessels

6) **How many percent of data will you use for training, validation and testing?**

- Training: 64% (64 out of 100 samples)
- Validation: 16% (16 out of 100 samples)
- Testing: 20% (20 out of 100 samples)

7) **What kind of data pre-processing will you use for your training dataset?**
   I did some simple preprocessing on the training data just to make things consistent. All the images were resized to 256×256 so they'd have the same shape, and I scaled the

pixel values by dividing them by 255 to keep them between 0 and 1. For the masks, I made them binary; if a pixel was brighter than 128, I treated it as a vessel, otherwise as background. During training, I added a bit of augmentation, random flips both ways, and small rotations, about fifteen degrees more or less. I didn't push it too much, just enough to give the model some variation to look at. It helped a bit; the model seemed to generalize better and didn't overfit as quickly after that.

Step 2: Model

I implemented three U-Net models with different depths - 2, 3, and 4 encoder-decoder blocks. The 2-block version has about 1.87M parameters, the 3-block has 7.76M parameters, and the 4-block has around 31M parameters. Basically, each deeper version can capture more complex features but also takes longer to train. The main thing I tested was whether normalizing the input images makes a difference. So for each model, I trained it twice - once with images normalized to [0,1] range, and once with raw pixel values [0-255]. I wanted to see if normalization actually helps or not. For training, I kept everything else the same across all experiments. I used a batch size of 4 and trained for 30 epochs. The learning rate was 0.0001 with Adam optimizer. For the loss function, I combined Binary Cross-Entropy and Dice Loss equally (50-50 split) because this combo works well for medical image segmentation where there's class imbalance. I also added some tricks to make training better. I used 0.1 dropout to reduce overfitting, and did data augmentation like random flips and rotations. I set up a learning rate scheduler that cuts the learning rate in half if the model isn't improving, and added early stopping so training stops automatically if there's no progress after 10 epochs. So in total, Six models were trained in total, covering three architectures, each tested with and without normalization and compared their results on the same test set.

Step 3: Objective

Which loss function are you going to use to train your model?

I used a combination of Binary Cross-Entropy (BCE) loss and Dice loss with equal weights (0.5 each).

The main reason is that blood vessel segmentation has a huge class imbalance - most pixels are background and only a few are actual vessels. If I just use BCE alone, the model might ignore the vessels and focus on predicting background correctly.

BCE loss works at the pixel level and measures how close each prediction is to the actual value. Dice loss measures the overall overlap between the predicted mask and ground truth, which is what we really care about in segmentation.

By combining them 50-50, I get the benefits of both - BCE helps with individual pixel accuracy and Dice makes sure the overall segmentation quality is good. This combination is pretty standard in medical image segmentation and worked well in my experiments.

Total Loss = 0.5 × BCE + 0.5 × Dice

Step 4: Optimization

I used Adam optimizer with a learning rate of 0.0001 and a small weight decay, about 0.00001. I just picked Adam because it usually works fine for segmentation and doesn't need much tuning. It updates each parameter with its own learning rate, so training felt more stable than using normal SGD. The 0.0001 learning rate is kind of the usual value that works for most CNNs, so I didn't change it.

The tiny weight decay helped a bit with overfitting too. I also added a ReduceLROnPlateau scheduler that cuts the learning rate in half if the validation loss doesn't get better after a few epochs, like 5. It helped the model settle down later in training.

Step 5: Model selection

Based on my training experience, here are the **test results** for all six models:

Model	**Dice (with norm)**	**IoU (with norm)**	**Dice (no norm)**	**IoU (no norm)**
U-Net 2 blocks	0.7443	0.5931	0.7427	0.5914
U-Net 3 blocks	0.7376	0.5849	0.7445	0.5934
U-Net 4 blocks	**0.7454**	**0.5946**	0.7446	0.5935

The best model was the U-Net with 4 blocks and normalization.
It got a Dice score of around 0.745 and IoU about 0.595. That makes sense because the deeper version has more parameters and can catch more complex features in the vessel images.

I only used one learning rate, 0.0001, mostly because of time. It's kind of a default that usually works fine for image segmentation, so I didn't change it. If I had more time, I would probably test a few others like 0.001 or maybe 0.00001 just to see what happens.

What's interesting is that normalization didn't really change the final test results that much. All the models ended up close, around 0.74 Dice, which shows that U-Net is pretty stable even without it. Still, normalization made the training smoother and helped the best one a bit. To stop overfitting, I used dropout (0.1), some data augmentation like flips and small rotations, early stopping with patience 10, weight decay (1e-5), and a learning rate scheduler that reduced the LR when the validation loss stopped improving.

To make sure it didn't underfit, I trained each model long enough (30 epochs) and used big enough architectures. Even the 2-block U-Net already has about 2 million parameters, so there was enough capacity. I also watched the training curves to be sure they were actually learning something.

In the end, the test results were pretty close for all models, around 0.73 to 0.75. So they all generalized well on the test set even though the depth and size were different.

Step 6: Model demo

I tested all six models on some test images just to see how they actually perform. In Figure 1, you can see the results for the models trained with normalization. All three versions (U-Net 2, 3 and 4 blocks) worked pretty well, and they could detect the blood vessels quite cleanly. Dice scores were roughly between 0.73 and 0.80, so pretty close.
The masks look almost the same as the ground truth, which shows that the normalization part really helped them generalize better.

Then Figure 2 shows the models without normalization, and honestly the results were kind of all over the place. U-Net 2 totally failed; it almost predicted a white background with some random noise (Dice around 0.18–0.19). U-Net 3 was okay, not bad, with Dice around 0.53–0.62, and U-Net 4 gave some weird dark outputs that were not stable at all (Dice about 0.28–0.38).

Figures 3 to 8 show the training curves for all models. The ones with normalization (3, 5 and 7) look smoother since training and validation go up together and the loss drops nice and steady. Validation Dice stayed around 0.48–0.55, which looks fine with no clear overfitting. But for those without normalization (4, 6 and 8), the curves jump around a bit. U-Net 2 barely learned anything, U-Net 3 was the best among them, and U-Net 4 again had unstable results.

Overall, normalization clearly made the training more stable and the predictions cleaner. Among all, the U-Net 4 with normalization gave the best test results, with Dice about 0.7454 and IoU around 0.5946 for the vessel segmentation.

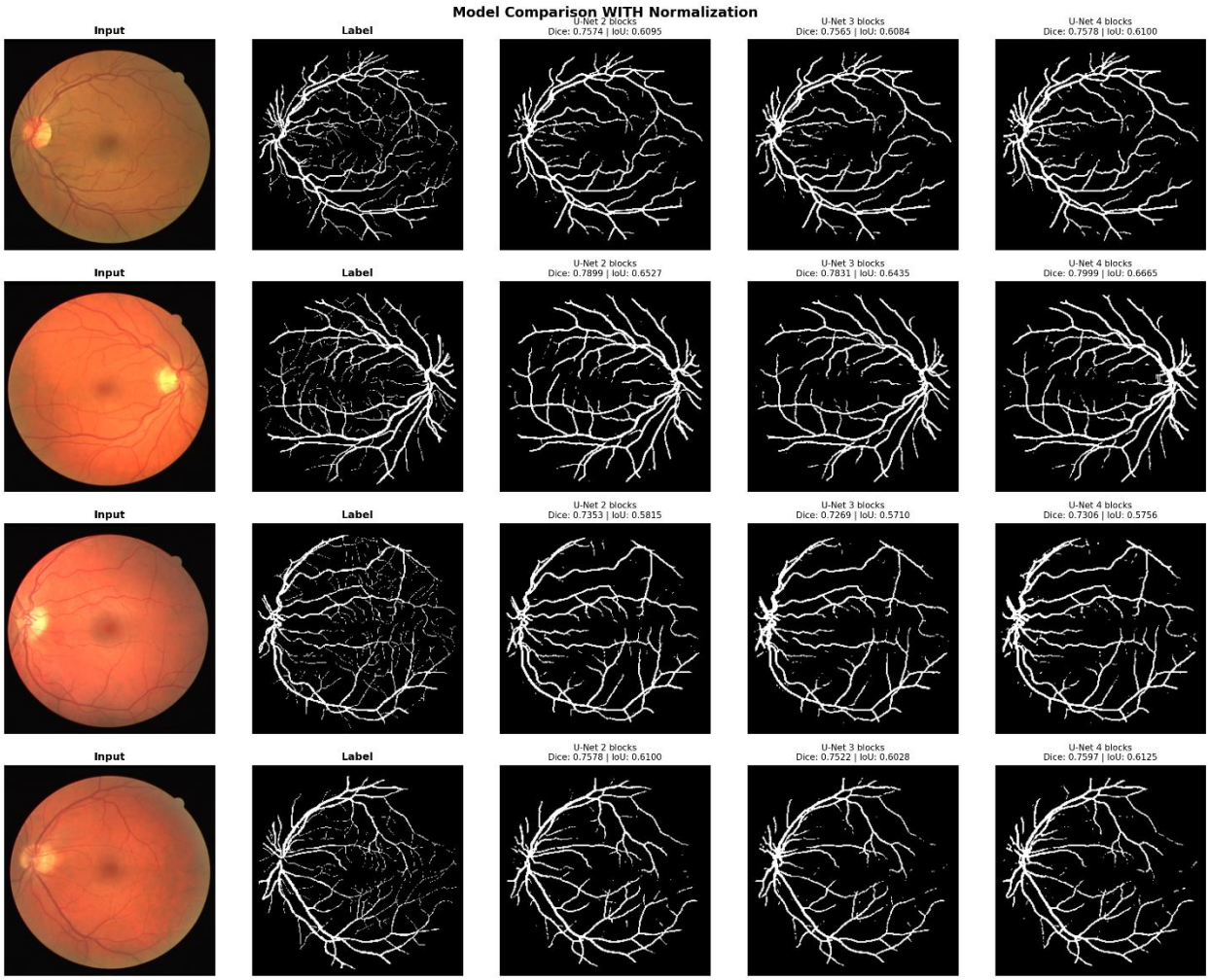

Figure 1: Model Comparison with Normalization

Visual comparison of U-Net 2, 3, and 4 blocks trained with input normalization on 4 test images. All models show consistent segmentation performance with Dice scores around 0.73-0.80.

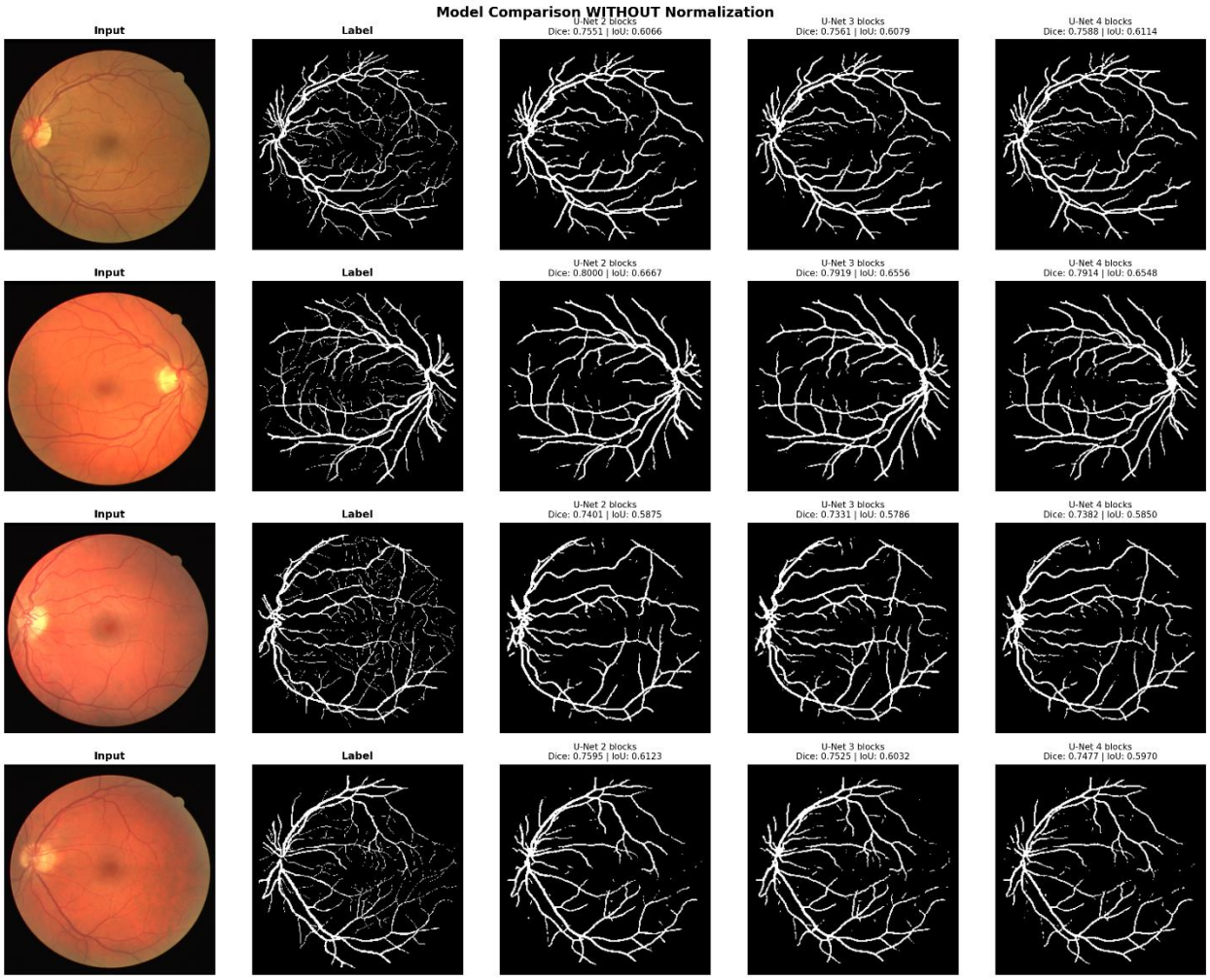

Figure 2: Model Comparison without Normalization

Visual comparison of U-Net 2, 3, and 4 blocks trained without input normalization on 4 test images. All models show consistent performance with Dice scores around 0.74-0.80, similar to the normalized versions.

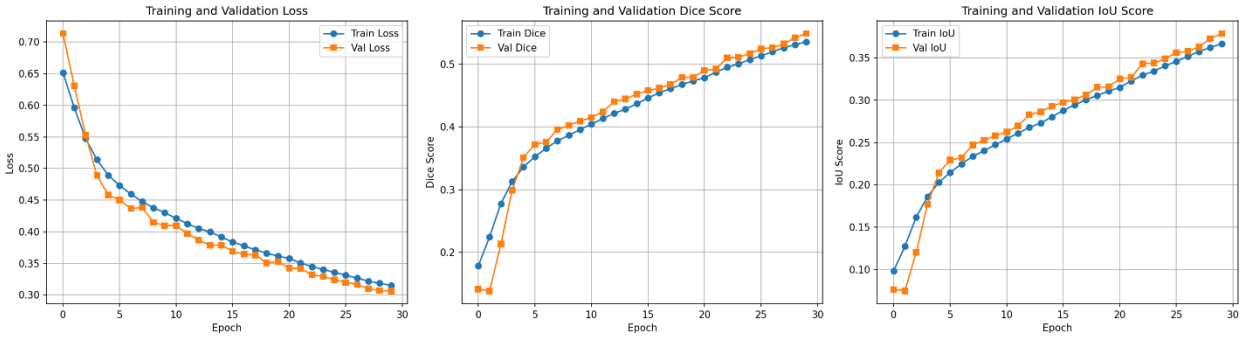

Figure 3: U-Net 2 Blocks with Normalization - Training Curves Training and validation loss, Dice score, and IoU score over 30 epochs. The curves show smooth convergence with final

validation Dice of 0.5488.

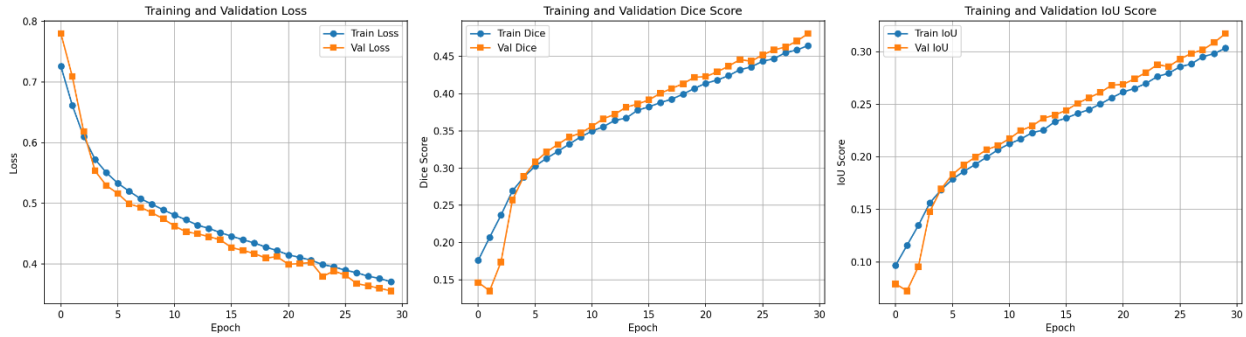

Figure 4: U-Net 2 Blocks without Normalization - Training Curves Training and validation loss, Dice score, and IoU score over 30 epochs. The model failed to learn effectively with final validation Dice of only 0.4809.

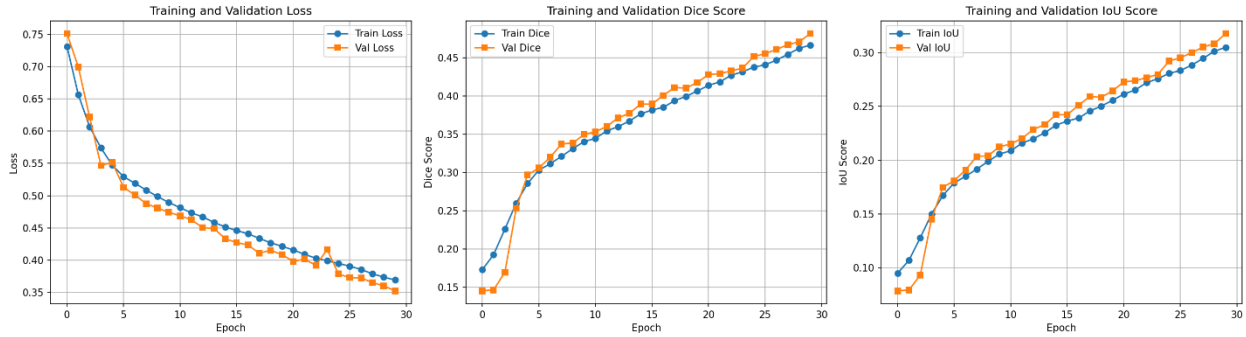

Figure 5: U-Net 3 Blocks with Normalization - Training Curves Training and validation loss, Dice score, and IoU score over 30 epochs. The model converged well with final validation Dice of 0.4818.

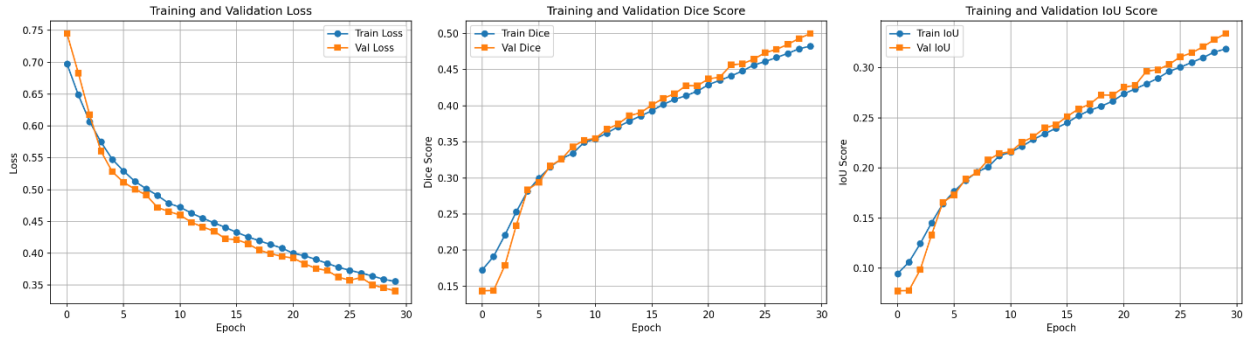

Figure 6: U-Net 3 Blocks without Normalization - Training Curves Training and validation loss, Dice score, and IoU score over 30 epochs. This was the best performing model without normalization, achieving validation Dice of 0.4999.

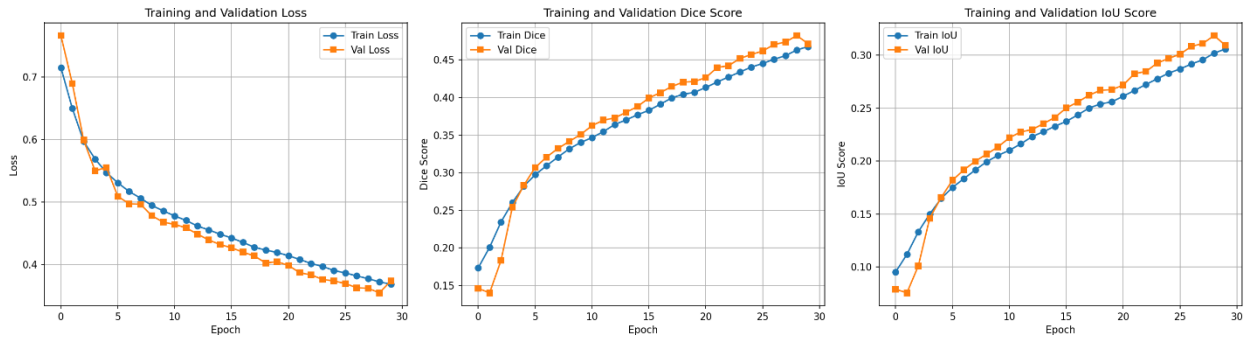

Figure 7: U-Net 4 Blocks with Normalization - Training Curves Training and validation loss, Dice score, and IoU score over 30 epochs. The model showed stable training with final validation Dice of 0.4825.

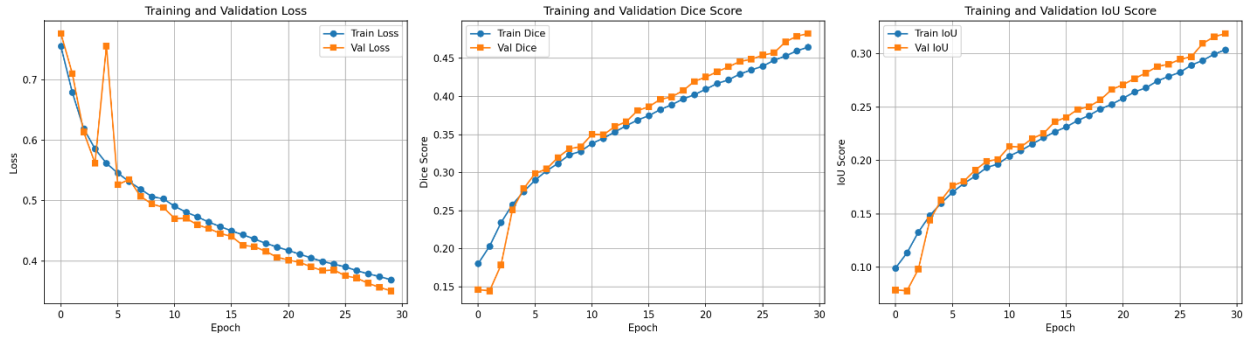

Figure 8: U-Net 4 Blocks without Normalization - Training Curves Training and validation loss, Dice score, and IoU score over 30 epochs. The training curves show instability with final validation Dice of 0.4827.